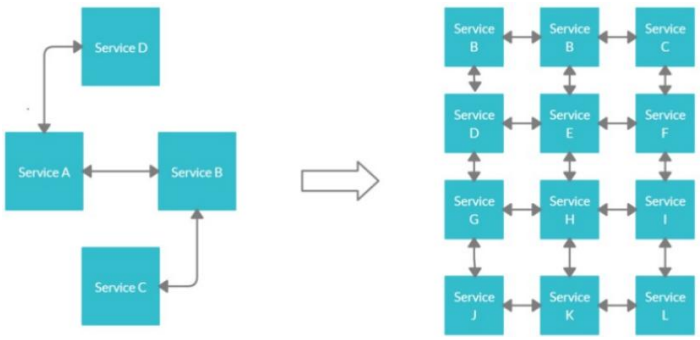
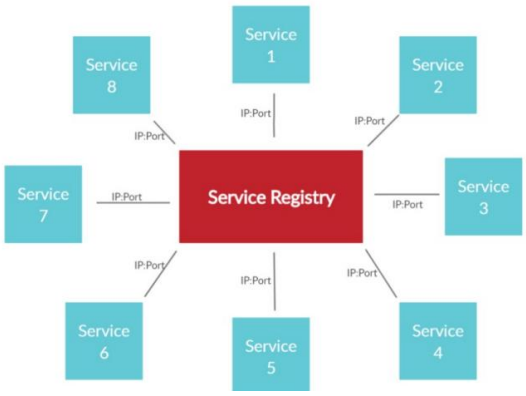
- Microservices are an architectural style where applications are developed as a collection of small, loosely coupled, independently deployable services.
- Comparison with Monolith
  - Monolith: tightly coupled, single code-base, single point of failure
  - Microservices: loosely coupled, multiple independent services
- Rest Client, Open Feign => these are some tools for api calling from one service to another.
- **Service Discovery**
  - In microservices, each service has to talk with the others to function as a *complete application*.
  - For this, they need to know each other's *network locations*.
  - Service discovery comes into play here, maintaining a record of service's locations, helping each other find each other and enabling communication.
- **Service Registry**
  - If we add one more service in out Microservice Architecture, all the services has to keep the address of new service.
  - If we add multiple services, then it'll not be properly maintaintable.
  - So, instead of keeping the address themselves, all the services register with a *Service Registry* so that if any new service is introduced, it can know about the other's addresses.
  - It's a *Centralized* service that registers all the services.
  - 

    (without service registry)

- 

  (with service registry)

- **Spring Cloud Eureka**
  - It's a REST based service which is primarily used for acquiring information about services that you would want to communicate with.
  - This REST service is also known as *Eureka Server*.
  - The services that register in Eureka Server to obtain information about each other are called *Eureka Clients*.
- **Eureka Server** in *Spring Boot*
  - Its another spring application service having *Eureka* dependency which will register all the services.
  - ```
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
    </dependency>
    ```
  - Enable the Eureka server
    ```
    @SpringBootApplication
    @EnableEurekaServer
    public class DiscoveryServiceApplication {
    ```
  - Disable the service registering for eureka service (by default it is enabled)
    ```
    eureka:
      client:
        fetch-registry: false
        register-with-eureka: false
    ```
    ```

- In the microservices, (because these are the eureka clients) add the *Eureka Client* dependency.

```
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
    </dependencies>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
```

- Register the eureka server in the client services

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

- Now when you run the eureka server then run the other services (eureka clients) they'll get registered in the eureka server. You can see in the eureka dashboard (localhost:8761 or whatever port you have configured)

How Eureka Works?

- **Service Registration**
  - A service registers itself with the Eureka Server upon startup.
- **Heartbeat**
  - The service sends heartbeats periodically to renew its lease with the Eureka Server.
  - Otherwise Eureka will think that the service is down.
- **Service Discovery**
  - Other services can query Eureka server to discover the location (IP and Port) of the registered service.
- **Health Check**
  - Eureka performs health checks to ensure that registered services are still healthy.
- **Eviction**
  - If a service stops sending heartbeats and its least expires, the Eureka evicts it from the registry.
- The auto-configure class for Eureka Server is *EurekaServerAutoConfiguration* .
  - This manages service registration, service registry, heartbeats, replication between servers.
- There is a *singleton* class *PeerAwareInstanceRegistryImpl* which is responsible for sending ***heartbeat*** to the Eureka Server.

- **NOTE**
  - **DiscoveryClient** is the main interface of *Spring Cloud* or you can say the rule book.
  - This has to be implemented by any *service registry* provider like *Netflix Eureka, Consul, or Zookeeper* ..etc.

    For example, Netflix Eureka implementation is: **EurekaDiscoveryClient**
  - Spring Cloud *doesn't* provide its own service registry implementation,

    Instead, it integrates with external registries like Netflix Eureka, Consul, or Zookeeper.

    For example, Spring Cloud Netflix provides auto-configuration and client libraries that allow Spring Boot microservices to register with a Eureka server and discover other services dynamically.
  - Netflix Eureka server is implemented in Java and usually runs as a Spring Boot application.

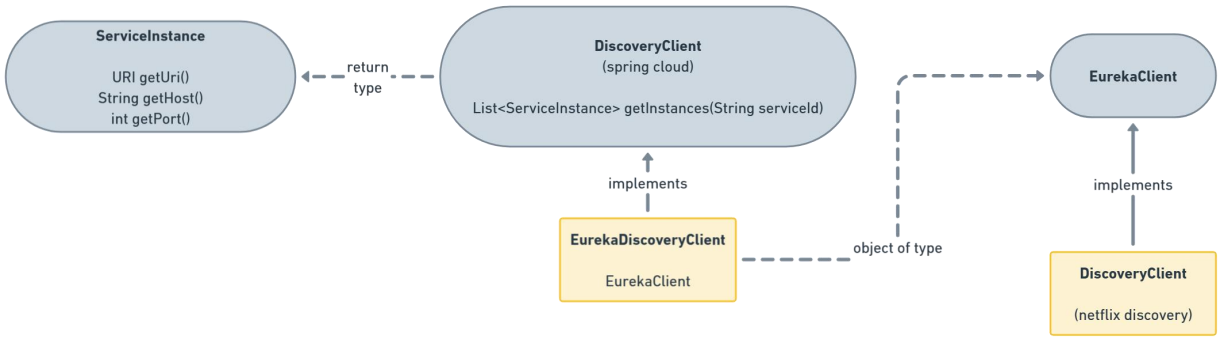
    Services written in other languages such as Node.js can still participate in service discovery by using Eureka client libraries like eureka-js-client (npm module) but the server itself typically runs on the JVM.
  - The Eureka clients get the IP and Port of the required service from the Eureka Server and trigger the request using **RestClient** (in case of Spring Boot applications).
  -
  -

- How to use Eureka?
  - There are 2 same interfaces present, named as **DiscoveryClient**, one belong to *spring cloud* and another belongs to *netflix*.
  - The flow of call happens like below

```
Spring Cloud DiscoveryClient (interface)
        ↓
EurekaDiscoveryClient (Spring adapter)
        ↓
Netflix DiscoveryClient (actual Eureka client)
        ↓
Eureka Server
```

  - *EurekaDiscoveryClient* (by *netflix*) implements *DiscoveryClient* (by *spring cloud*)
  - *EurekaDiscoveryClient* contains an object of type *EurekaClient*
  - This *EurekaClient* is implemented by *DiscoveryClient* (by *netflix*).
  - `List<ServiceInstance> instances = discoveryClient.getInstances( serviceId: "order-service");`

    this *serviceId* is the name of spring boot application that is being mentioned in *application.yml* or *application.properties* file.



  - NOTE: You may see ***RestClient*** in Eureka classes but it doesn't use RestClient. It uses its own HTTP client internally. The netflix client uses **EurekaHttpClient**

```
@GetMapping("/fetchOrders")
public String fetchFromOrderService() {
    ServiceInstance orderService = discoveryClient.getInstances( serviceId: "order-service").getFirst();

    return restClient.get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: orderService.getUri() + "/api/v1/orders/helloOrders") capture of ?
            .retrieve() ResponseSpec
            .body( bodyType: String.class);
}
```

  - This is how you can implement microservices.

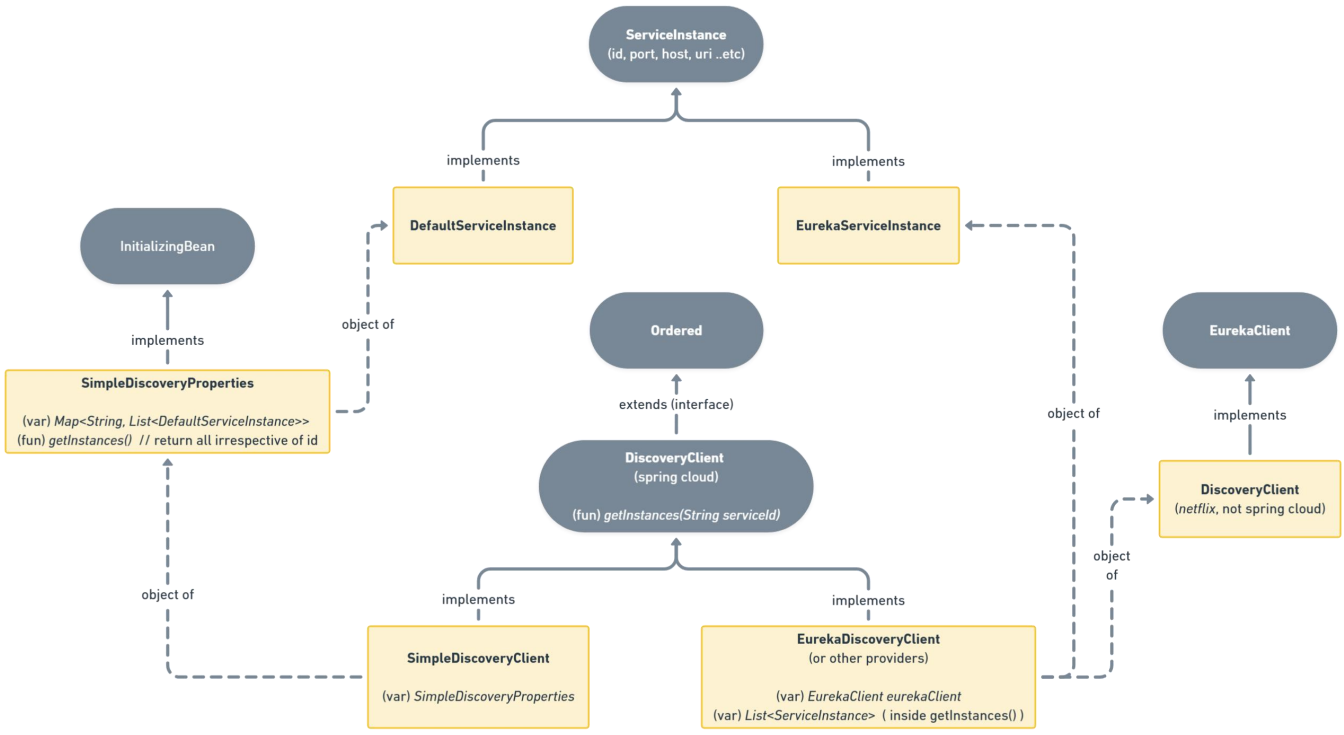
    Eureka is only for the IP and Service management. **RestClient** is to use that url and trigger HTTP requests and get response.

```yaml
spring:
  application:
    name: <your-service-name>  # Name of your microservice

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/  # URL of the Eureka Server where the client should register
    fetch-registry: true      # (usually true for clients)
    register-with-eureka: true  # (usually true for clients)

  instance:
    prefer-ip-address: true     # Use IP instead of hostname for registration (useful for containers)
    lease-renewal-interval-in-seconds: 10  #  (default is 30)
    lease-expiration-duration-in-seconds: 30  #  (default is 90)
```

DiscoveryClient complete implementations



- 2 DiscoveryClient are there, one is from Spring Cloud, another is from Netflix. Both serves different purposes.
  - Spring cloud's DiscoveryClient (**interface**) is get the instances according to service id and like those.
  - Netflix's DiscoveryClient (**class**) is the helper to fetch the things from Eureka Service Registry.

    You can say, Spring Cloud's DiscoveryClient is abstraction, and Netflix's DiscoveryClient is implementation.
- Here, this **DiscoveryClient** is of Spring Cloud.

  It has 2 implementations
  - **SimpleDiscoveryClient**
  - **EurekaDiscoveryClient** (because I am using Eureka)
- SimpleDiscoveryClient
  - Here, all the services details has to be given in the application.yml or application.properties file.
  - It is service specific, all the services need to store the details of other services to connect with those.
  - It just store the things in a *HashMap*.

- It uses **DefaultServiceInstance** (which implements **ServiceInstance**) via **SimpleDiscoveryProperties**.
  - *SimpleDiscoveryProperties*'s *getInstances()* method returns all instances irrespective of service id.
-

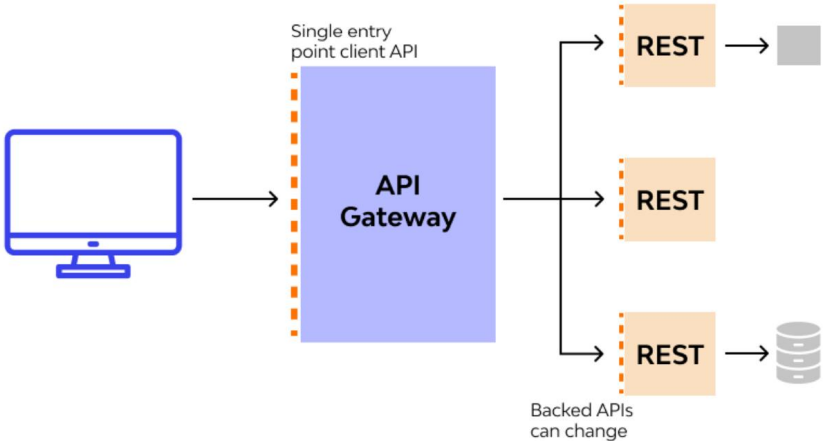
```
spring:
  cloud:
    discovery:
      client:
        simple:
          instances:
            ORDER-SERVICE:
              - uri: http://localhost:8081
              - uri: http://localhost:8082
```

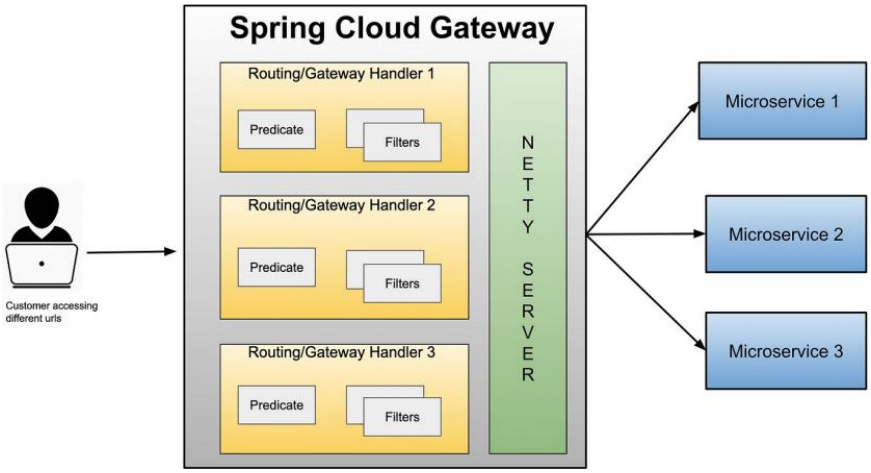
- **EurekaDiscoveryClient**
  - It uses EurekaClient (which is implemented by Netflix's DiscoveryClient) to get the instances according to service id.
  - In this case, no need to store the ip and other details of the services in each services, just keep a *Eureka Server* (centralized) and the details will be fetched from here only.
  -

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

-
- F
-

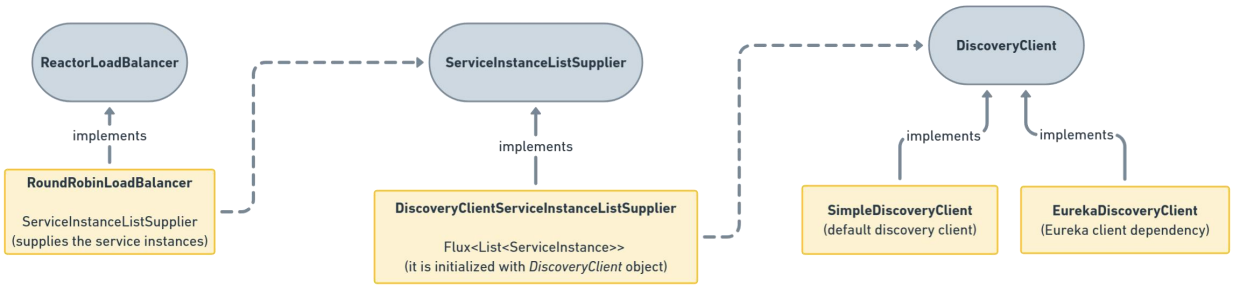
Spring Cloud API Gateway

- Description
  - APIs are a common way of communication between applications. In case of microservice architecture, there will be a number of services and the client has to know the host names of all underlying applications to invoke them.
  - To simplify the communication, we prefer a component between client and server to manage all API requests called API gateway. Additionally, we can have other features which includes:
    - *Security*: authentication, authorization
    - *Routing*: routing, request/response manipulation, circuit breaker
    - *Observability*: metric aggregation, logging, tracing
  - 

  - There are multiple options out there, but the most optimized in case of Spring is Spring Cloud Gateway.
  - 

- Implementation of API gateway
  - Keep 2 dependencies: Eureka Discovery Client, Gateway (Spring Cloud) (Eureka Client because API gateway also needs to connect with the Eureka server to get the details about other services)
  - If you trace the *pom.xml* dependencies "spring-cloud-starter-gateway-server-webmvc" ---- "spring-cloud-gateway-server-webmvc" ---- "spring-cloud-loadbalancer".
- Spring Cloud Gateway **route**
  - The destination that we want a particular request to route to.
  - It comprises of destination URI, a condition that has to satisfy - Or in terms of technical terms, *Predicates,* and one or more *filters*.
- Spring Cloud Gateway **predicate**
  - This is literally a condition to match, I.e. kind of "if" condition.
  - - Path=/api/v1/orders/**
  - - Method=GET
  - - Header=User-Agent, Mozilla/*
- F

- Some confusion regarding Spring Cloud API Gateway
  - In case of OAuth or payment integration or caching (redis or something else), etc.. spring has library which delegates the calls to the specific services.
  - But, Spring Cloud API Gateway is a stand alone service that implements API gateway and integrated Load Balancer.
    - Just like ***nginx*** provides API gateway and load balancer, Spring Cloud also provides the same ***by its own***.
  - Without *Eureka Client* dependency also it'll work. But it'll not be dynamic.



- Working of Gateway and Load Balancer
  - Lets say you have configured the application.yml file like below

```
spring:
  cloud:
    discovery:
      client:
        simple:
          instances:
            ORDER-SERVICE:
              - uri: http://localhost:8081
              - uri: http://localhost:8082
```

    - In this case you have directly mentioned the end points of the instances of order-service (same service can have multiple instances)
    - spring docs for this yaml structure
      spring.cloud.discovery.client.simple.instances.service1[0].uri=http://s11:8080
    - So, here the ***SimpleDiscoveryClient*** will return all these instances to the ***RoundRobinLoadBalancer*** , out of which it'll choose a instance and delegates the request to that.
      (*RoundRobinLoadBalancer* class has a ***choose()*** method)
    - Internally it calls like **lb://ORDER-SERVICE** which triggers the load balancer.

- Here the flow will be like ***client -- gateway --- discovery client --- instances --- choose (load balance) --- delegate to instance***

```
Gateway
 ↓
Spring Cloud LoadBalancer
 ↓
DiscoveryClient (Eureka / SimpleDiscoveryClient)
 ↓
[instance1, instance2, instance3]
 ↓
Round Robin selection
```

- In case of the below, load balancing will not happen and you can only give one end-point per service

```
spring:
  cloud:
    gateway:
      routes:
      - id: after_route
        uri: https://example.org
```

```
routes:
  - id: users
    uri: http://user-service:8081
    predicates:
      - Path=/users/**

  - id: orders
    uri: http://order-service:8082
    predicates:
      - Path=/orders/**
```

- Its flow will be like ***client -- gateway -- delegate to instance***

```
Client
  ↓
Gateway
  ↓
https://example.org
```

- Now, if you are using **Eureka Client** then you don't need to hard-code the end points, because instead of **SimpleDiscoveryClient** here EurekaDiscoveryClient will be used which will get the end points of the instances from the Eureka Server.

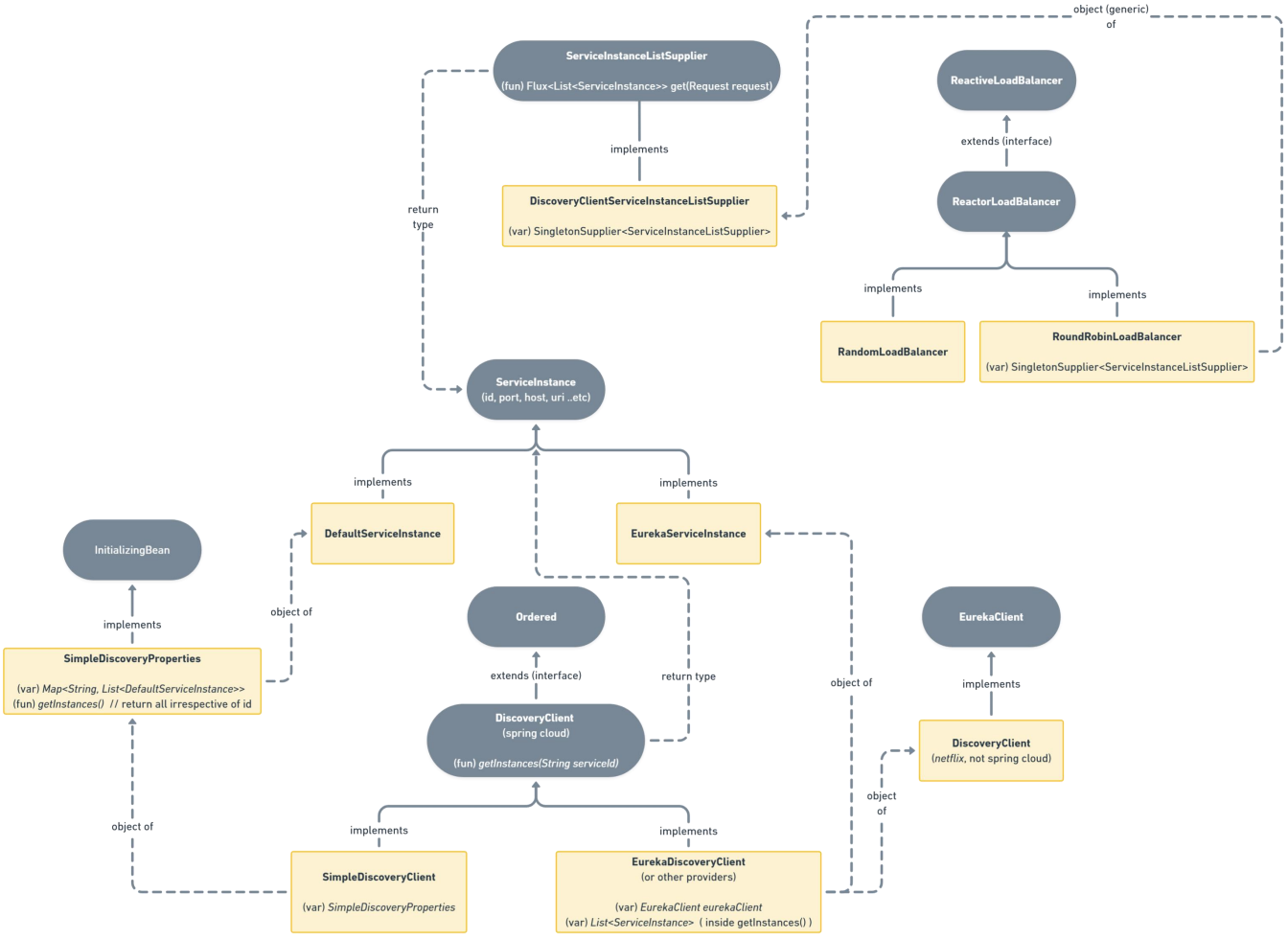
- ```
  spring:
    cloud:
      gateway:
        routes:
          - id: order-service
            uri: lb://ORDER-SERVICE
            predicates:
              - Path=/api/v1/orders/**
            filters:
              - AddRequestHeader=X-Custom-Header, Anuj
          - id: inventory-service
            uri: lb://INVENTORY-SERVICE
            predicates:
              - Path=/api/v1/inventory/**
          - id: no-service
            uri: lb://NO-SERVICE
            predicates:
              - Path=/api/v1/no-service/**
            filters:
              - AddRequestHeader=X-Custom-Header, Anuj
              - RedirectTo=302, https://codingshuttle.com
  ```

- **predicates**
  - Its nothing but a condition, if it passes then only the request will be forwarded otherwise not.
  - In case of *predicates* ***Path***, if the request end point matches this path, then only the request will be forwarded otherwise not.
    - For example:
      - Path=/orders/**
      - if request path is /orders/123 then it'll match and it'll be forwarded.
  - Different types of predicates are there like Path, Method, Header, Cookie, Host ..etc
  - ```
    Client request
          ↓
    Gateway checks all routes
          ↓
    Predicates evaluated
          ↓
    Matching route selected
          ↓
    Request forwarded to URI
    ```
  - Don't think it is used for authorization, for example *Cookie=mycookie,mycookievalue* , it'll just check if cookie having name

"mycookie" is present and having the value "mycookievalue", if this condition passes then it'll forward. It'll not validate the cookie value.

- To configure the API Gateway
  - ```
    cloud:
      gateway:
        routes:
          - id: order-service
            uri: lb://ORDER-SERVICE
            predicates:
              - Path=/orders/**
          - id: inventory-service
            uri: lb://INVENTORY-SERVICE
            predicates:
              - Path=/inventory/**
    ```
    - order-service has context path: /orders, and the main order controller has path "core";
      so, the path is "/orders/core"
    - Similarly inventory-service has path: "/inventory/products"
    - So, this will work. If we open *localhost:8080/orders/core* or something like that, then it'll forward the request to the specific service.
  - But, sometimes we want versioning, like "/api/v1" or "/api/v2"; but we don't want to give these as context path in the respective services. We want to control these from gateway itself.
    - ```
      cloud:
        gateway:
          routes:
            - id: order-service
              uri: lb://ORDER-SERVICE
              predicates:
                - Path=/api/v1/orders/**
              filters:
                - StripPrefix=2
            - id: inventory-service
              uri: lb://INVENTORY-SERVICE
              predicates:
                - Path=/api/v1/inventory/**
              filters:
                - StripPrefix=2
      ```

- Here, we can use the filter *StripPrefix*, **2** means it'll remove the first 2 i.e. "api" and "v1" and forward the remaining i.e. http://localhost:<service_port>/orders/** (it'll not include /api/v1)
- Filters are executed between gateway and services.
- Filters are used to modify the request at gateway level before forwarding this to the services.



- This is how, **Load Balancer** is linked with the service instance.
  - Most commonly (and default), **RoundRobinLoadBalancer** is being used which fetch all the instances (objects of ServiceInstance) and choose among them, and delegates the HTTP call to that service.
  -
- f

OpenFeign (by Netflix)

- Spring Cloud **OpenFeign** is a declarative HTTP client library for building RESTful microservices.

  It integrates seamlessly with Spring Cloud and simplifies the development of HTTP clients by allowing you to create interfaces that resemble the API of the target service.

  It abstracts away much of the boilerplate code typically associated with making HTTP requests, making your code-base cleaner and more maintainable.

  - Its basically replacement of ***RestClient*** so that we don't need to write much code.

- How to implement? -----

  - Annotate the main application with **@EnableFeignClients**
  - Create a *interface* and annotate it with **@FeignClient**

```
@FeignClient(name = "order-service", path = "/orders")
public interface OrdersFeignClient {

    @GetMapping("/core/helloOrders")  no usages
    String helloService();
}
```

  - If you give *path* in **@FeignClient** annotation, then it'll take that path as prefix for all the methods inside that (just like @RequestMapping)
  - And the methods will be just like controller methods with annotation ***@GetMapping, @PostMapping, @PutMapping*** .. etc.
  - **NOTE**
    - the feign client should be *interface*.
    - Name should same as the service name. Here **order-service**
    -
  -
  -

- One drawback in using **RestClient** was the load balancing.

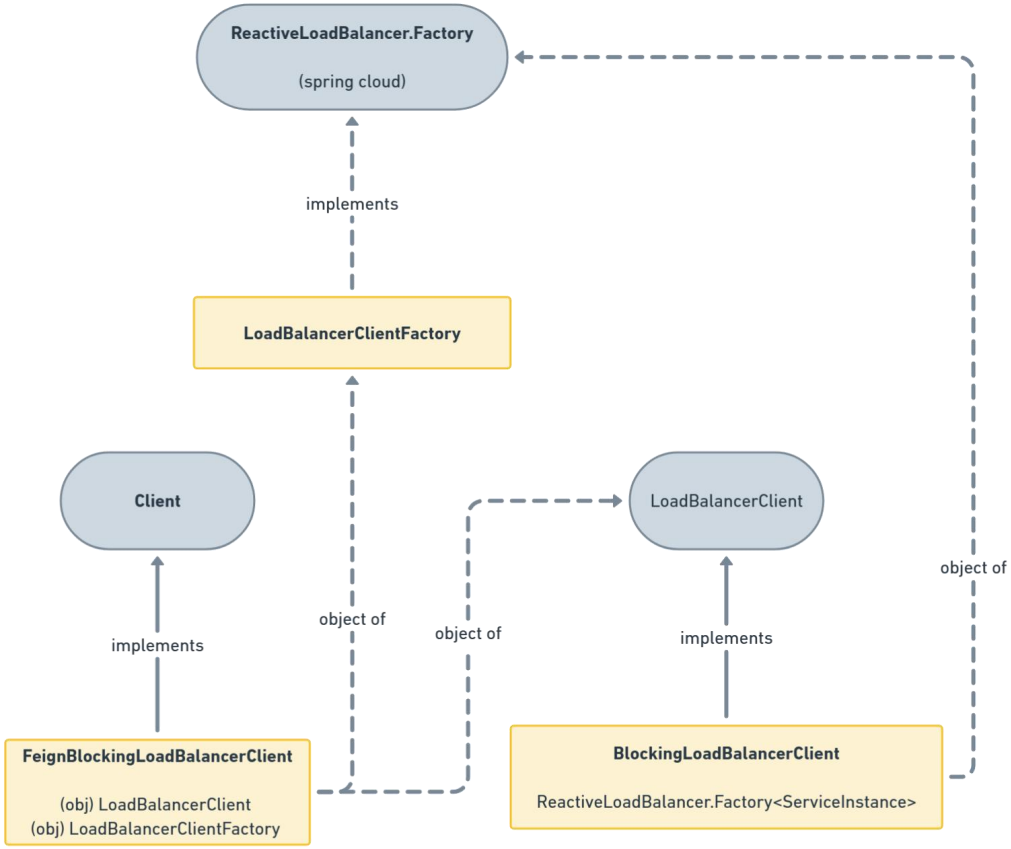
```
@GetMapping("/fetchOrders")   Alok Ranjan Joshi *
public String fetchFromOrderService() {
    ServiceInstance orderService = discoveryClient.getInstances( serviceId: "order-service").getFirst();

    return restClient.get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: orderService.getUri() + "/orders/core/helloOrders") capture of ?
            .retrieve() ResponseSpec
            .body( bodyType: String.class);
```

  - We were using it like this, we had to get a instance of that particular service to trigger our request. Which is not proper.
  - If we have multiple instances of a service then we need a load balancer here as well which will send the request to a instance having less congestion.
  - **FeignClient** solved this issue.

- Internal working & Load balancing of **FeignClient** ----------

  - Feign contains a class **FeignBlockingLoadBalancerClient** which manages the load balancing part.




```
public class FeignBlockingLoadBalancerClient implements Client {  no usages
    private static final Log LOG = LogFactory.getLog( clazz: FeignBlockingLoadBalancerClient.class);
    private final Client delegate;
    private final LoadBalancerClient loadBalancerClient;
    private final LoadBalancerClientFactory loadBalancerClientFactory;
    private final List<LoadBalancerFeignRequestTransformer> transformers;
```

- **FeignBlockingLoadBalancerClient** gets the *service id* (order-service, inventory-service ..etc) and get the instances using that *LoadBalancerClientFactory* object.
  - **ReactiveLoadBalancer.Factory** contains ***getInstances()*** method and this **ReactiveLoadBalancer** is implemented by ***RoundRobinLoadBalancer*** which is being used by the spring cloud API gateway.
- Then it choose the instance by the **LoadBalancerClient** object present inside it.
- ```
  ServiceInstance instance = this.loadBalancerClient.choose(serviceId, lbRequest);
  ```
- So in our case, **2** load balancing are happening
  - Client to the service

    Client → API Gateway → Route match (/inventory/**) → Spring Cloud LoadBalancer (Gateway) → select inventory-service instance → inventory-service

    ```
    Client → API Gateway → Route match (/inventory/**) →
    Spring Cloud LoadBalancer (Gateway) →
    select inventory-service instance → inventory-service
    ```
  - Service to Service

    inventory-service → Feign Client → FeignBlockingLoadBalancerClient → Spring Cloud LoadBalancer → DiscoveryClient → Eureka registry → get order-service instances → select instance → HTTP call → order-service

    ```
    inventory-service → Feign Client → FeignBlockingLoadBalancerClient →
    Spring Cloud LoadBalancer → DiscoveryClient → Eureka registry →
    get order-service instances → select instance → HTTP call → order-service
    ```
- Now the question is how **FeignClient** get the instance even if we have not configured inside application.yml or application.properties.
  - In every Eureka client, we configure only this much

    ```
    eureka:
      client:
        service-url:
          defaultZone: http://localhost:8761/eureka
    ```
  - And in case of RestClient, we used to get the instance using service name

    ```
    ServiceInstance orderService = discoveryClient.getInstances( serviceId: "order-service").getFirst();

    return restClient.get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: orderService.getUri() + "/orders/core/helloOrders") capture of ?
            .retrieve() ResponseSpec
            .body( bodyType: String.class);
    ```
  - So, here 2 phase was there: 1. get the instance, 2. trigger the request
    ```

- **FeignClient** also does the same, in the FeignClient config interface we give the service-id as the name

```
@FeignClient(name = "order-service", path = "/orders")
public interface OrdersFeignClient {
```

- So, **FeignClient** get the instances using this service-id and use the load balancer to choose the appropriate instance.
- Then it trigger the request.

```
@GetMapping("/core/helloOrders")
String helloService();
```

the path is mentioned here

- Calling a ***put controller*** of a service from another service which contains **@RequestBody** as well.
  - In my case, I'll create a order in ***Order*** service, and I'll send the list of product and quantity list to the ***Inventory*** service so that it'll reduce the stock of products from database.
  - For this, whatever names and types are present in the @RequestBody DTO in **Inventory** controller, I'll need to send exactly those from **Order** service.
  - Inventory service DTO

```
public class OrderRequestDto {
    private List<OrderRequestItemDto> items;
}
```

```
public class OrderRequestItemDto {
    private Long productId;
    private Integer quantity;
}
```

  - Order service DTO

```
public class OrderRequestDto {
    private Long id;
    private List<OrderRequestItemDto> items;
    private BigDecimal totalPrice;
}
```

```
public class OrderRequestItemDto {  1 us
    private Long id;  no usages
    private Long productId;  no usages
    private Integer quantity;  no usages
}
```

- Here if you see, in order service's DTO, some extra fields are there but that is not a problem. All the fields that are being required in the inventory service must be there in order service's DTO.

  Just like, if you give extra some key values in json payload in postman doesn't cause any issue but giving less values might cause issue.
-
-

Circuit Breaker, Rate Limiting, Retry with Resilience4J

- Resilience4J is a lightweight, standalone library designed for implementing resilience patterns in Java applications, particularly those using microservices architecture.
  It provides mechanisms to handle failures gracefully and ensures that services remain responsive under failure conditions.
  (another library similar to Resilience4J is *Netflix Hystrix*)
  It's key features are
  - Retry
  - Rate Limiter
  - Circuit Breaker
- Dependencies
  - You'll get 2 dependencies, one from *Resilience4J* directly and another from *Spring Cloud*.

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-all</artifactId>
    <version>${resilience4jVersion}</version>
</dependency>
```

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

  - ***Spring cloud*** version is a abstraction layer on top of the core Resilience4J dependency.
    Because it should accommodate other circuit breaker providers as well like Sentinel, Spring Retry.
  - If you include *Resilience4J* dependency directly then you are bound to use this only, you can migrate to some other provider in future.
    So, you should use *Spring Cloud* dependency.
  - Another thing is, Spring Cloud provides auto-configuration of those in Spring Boot application.
  - If you don't want the auto-configuration then you can set the below in application.properties or application.yaml file.

```
spring.cloud.circuitbreaker.resilience4j.enabled=false
```

  - spring cloud resilience4j docs

- If you trace *Spring Cloud* dependency, you'll find the core Resilience4J dependency.

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-circuitbreaker</artifactId>
  <version>2.3.0</version>
  <scope>compile</scope>
</dependency>
```

- In all the annotations like ***@Retry***, ***@RateLimiter***, ***@CircuitBreaker*** , same arguments are required which are: **name, fallbackMethod**

```
public @interface Retry {
    String name();

    String fallbackMethod() default "";
}
```

```
public @interface RateLimiter {
    String name();

    String fallbackMethod() default "";

    int permits() default 1;
}
```

```
public @interface CircuitBreaker {
    String name();

    String fallbackMethod() default "";
}
```

  - Name is -------- todo
  - **fallbackMethod** is the method which will be called after it give ups.
    That fallback method should have *same number of arguments, same types of arguments, same return type*. [fallback method name should be different of course]
    One extra argument will be there which is of type **Throwable** in the fallback method.

- Its *not bound to microservices* architecture, you can use these for any type of exceptions.
- [The properties to be auto-configured docs]

- **Retry**
  - Resilience4J can help write logic for retry when a service call fails.

    This retries failed operations a certain number of times before giving up.

```
@Retry(name = "inventoryRetry", fallbackMethod = "createOrderFallback")  1 usage  Alok
public OrderRequestDto createOrder(OrderRequestDto orderRequestDto) {...}

public OrderRequestDto createOrderFallback( @NotNull OrderRequestDto orderRequestDto,
                                            @NotNull Throwable throwable) {
    log.info("Falling back to create order with ID: {} with error: {}",
            orderRequestDto.getId(), throwable.getMessage());

    return new OrderRequestDto();
}
```

  - The fallback method has same return type, same type of arguments, with **Throwable** type of argument.
  - Then you can configure the things in application.yaml or application.properties file.

    Like max retry, and all.

```
resilience4j:
  retry:
    instances:
      inventoryRetry:
        maxRetryAttempts: 3
        waitDuration: 5s
```

  - That **name** field of ***@Retry*** is used here, you can set different configs for different *names*.
- F